

Builder, observer og animator



Undervisning
<https://www.youtube.com/watch?v=uNW05z-OwWM>

BUILDER PATTERN



Builder

- ▶ Builder Pattern separerer konstruktionen af et komplekst objekt fra dets repræsentation.
 - ▶ Dette gør det muligt at bruge samme konstruktionsproces til at lave forskellige varianter af objektet.
- ▶ **Hvornår er Builder Pattern nyttigt?**
 - ▶ Hvis et objekt har mange konfigurationsmuligheder, kan det forenkle skabelsen af objekter.
 - ▶ I stedet for mange forskellige constructors bruger vi en Builder, som gradvist modtager parametre.
 - ▶ Til sidst returneres det færdige objekt med den ønskede konfiguration.
- ▶ **Eksempel:** I stedet for at have flere constructors med forskellige parametre, laver vi én Builder, der håndterer opsætningen trin for trin.

Builder vs traditionel constructor

```
public class Character
{
    public string Name { get; }
    public string Class { get; }
    public int Health { get; }

    public Character(string name, string characterClass, int health)
    {
        Name = name;
        Class = characterClass;
        Health = health;
    }
}

// Bruger skal altid give ALLE parametre
Character warrior = new Character("Warrior King", "Warrior", 150);
```

Ulemper:

- Hvis vi vil tilføje flere parametre, **skal vi oprette endnu en constructor**.
- Vi **kan ikke lave et delvist færdigt objekt** – alt skal defineres med det samme.
- Skal have alle parametre for at kunne oprette

```
public class CharacterBuilder
{
    protected Character character = new Character();
    public CharacterBuilder SetName(string name)
    {
        character.Name = name;
        return this;
    }
    public CharacterBuilder SetHealth(int health)
    {
        character.Health = health;
        return this;
    }
    public Character Build()
    {
        return character;
    }
}

// Vi kan nu oprette en karakter med KUN nogle af sine dele
CharacterBuilder builder = new CharacterBuilder();
Character halfBuiltCharacter = builder.SetName("Mysterious
Stranger").Build();
Console.WriteLine($"Created: {halfBuiltCharacter.Name}, Health:
{halfBuiltCharacter.Health}"); // Health er stadig default (0)
```

• Fordele ved Builder Pattern:

- **Fleksibilitet** → Vi kan vælge **kun at tilføje nogle attributter** og stadig få et funktionelt objekt.
- **Fluent interface** → Vi kan bygge objektet med **chained metodekald**, hvilket gør koden mere læsbar.
- **Letere vedligeholdelse** → Hvis vi vil tilføje nye parametre, **kan vi gøre det uden at ændre eksisterende kode**.

Builder VS Factory

▶ Builder:

- ▶ Bruges til at konstruere komplekse objekter step by step.
- ▶ Objektet kan ikke skabes i ét enkelt step, men bygges gradvist.
- ▶ Ofte brugt til at sammensætte flere komponenter (composites).
- ▶ Bruges fx til Player, hvor vi gradvist tilføjer komponenter som SpriteRenderer, PlayerController, osv.
- ▶ Ville også give mening til Level-generation, hvor vi trin for trin opbygger en scene med objekter, fjender og miljø.

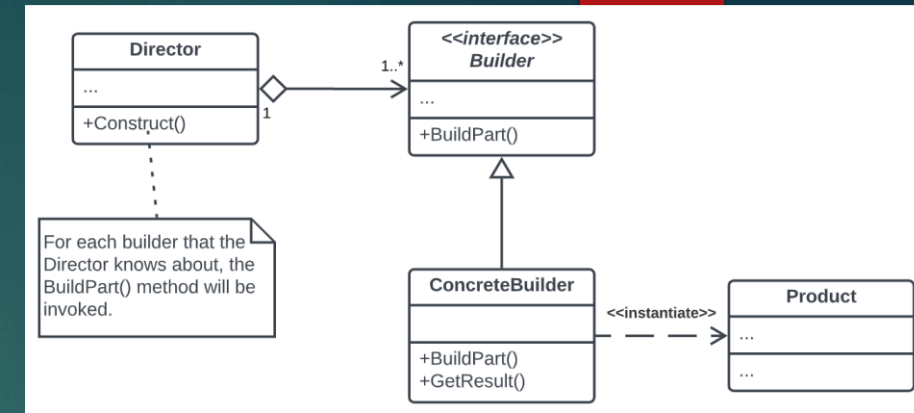
▶ Factory:

- ▶ **Bruges, når vi hurtigt skal skabe mange ens objekter.**
- ▶ Alle objekter har **samme grundstruktur**, men kan have **forskellige parametre**.
- ▶ Objektet **returneres med det samme** – ingen step-by-step konstruktion.
- ▶ Bruges typisk til **Enemies, projektiler (Lasers), pickups, power-ups**, hvor vi **bare skal have en ny instans hurtigt**.

Hvordan supplerer builder de factory?

- ▶ Factory Pattern kan bruge en Builder til at oprette objekter med en kompleks opsætning.
- ▶ **Eksempel:**
 - ▶ En LevelFactory kan bruge en LevelBuilder til at generere et færdigt level.
 - ▶ En EnemyFactory kan bruge en EnemyBuilder, hvis fjender har forskellige AI-opførsler og stats.
- ▶ Factory er hurtig og masseproducerer objekter, mens Builder giver os kontrol og fleksibilitet!

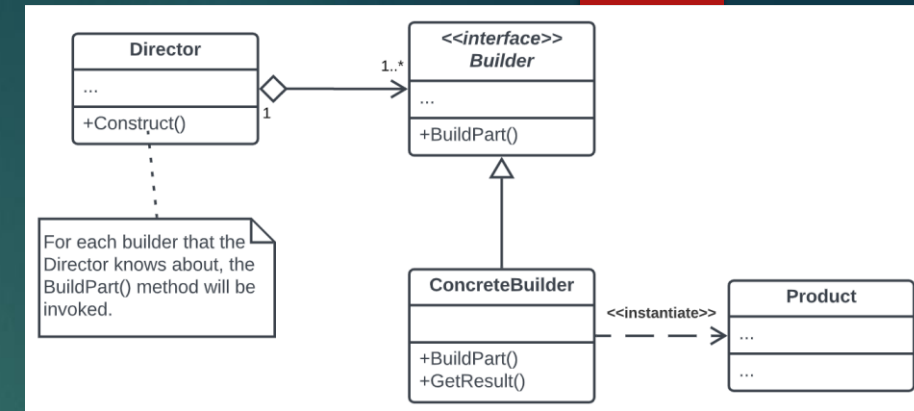
Builder



► Overordnet består et Builder Pattern af fire hovedelementer:

- **Builder Interface**
 - Definerer de forskellige trin, som kan bruges til at bygge objektet.
- **Concrete Builder**
 - Implementerer interfacet og udfører de specifikke trin i byggeprocessen.
- **Product**
 - Det objekt, der bliver oprettet af Builderen.
- **Director**
 - En valgfri klasse, der bruger en Builder til at returnere et færdigt objekt ud fra en fast opskrift.

Builder Interface



► Builder Interface

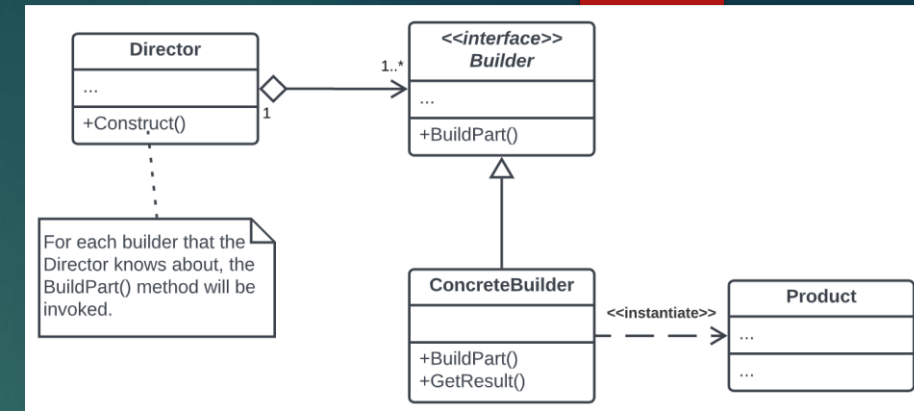
- Definerer de trin, der skal bruges til at bygge et objekt.
- Sikrer, at alle builders følger samme struktur.
- Tillader forskellige implementeringer af samme byggelegik.

```
public interface ICharacterBuilder
{
    ICharacterBuilder SetName(string name);
    ICharacterBuilder SetHealth(int health);
    ICharacterBuilder SetClass(string characterClass);
    Character Build();
}
```


Product

► Product

- Det objekt, der bliver bygget af Concrete Builder.
- Indeholder de egenskaber, der er sat via Builderen.
- Er færdigt, når Build() kaldes.



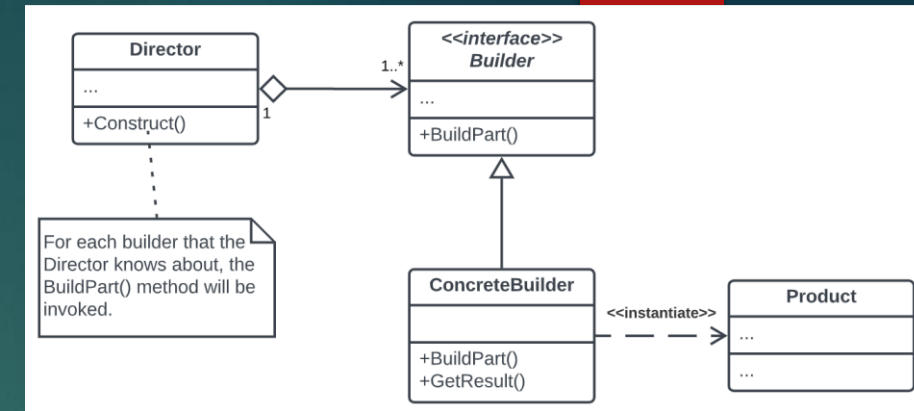
```
public class Character
{
    public string Name { get; set; }
    public string Class { get; set; }
    public int Health { get; set; }
}
```

Concrete Builder

► Concrete Builder

- Implementerer Builder Interface.
- Opbygger objektet trin for trin.
- Holder styr på, hvilke dele objektet skal have.
- Giver fluent interface:

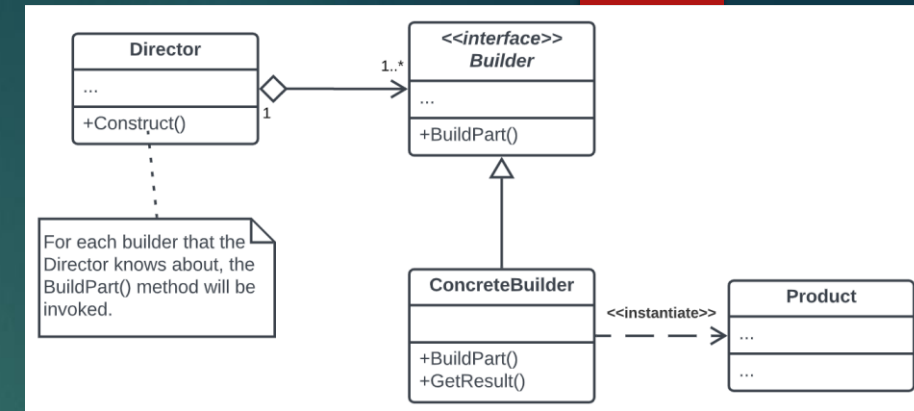
```
CharacterBuilder characterBuilder;  
Character kajChar = characterBuilder.SetName("kaj").SetHealth(100).Build();
```



```
public class CharacterBuilder : ICharacterBuilder
{
    private Character character = new Character();
    public ICharacterBuilder SetName(string name)
    {
        character.Name = name;
        return this;
    }
    public ICharacterBuilder SetHealth(int health)
    {
        character.Health = health;
        return this;
    }
    public ICharacterBuilder SetClass(string classType)
    {
        character.ClassType = classType;
        return this;
    }

    public Character Build()
    {
        return character;
    }
}
```

Director



► Director

- Holder en reference til en Builder og styrer byggeprocessen.
- Sikrer, at byggeprocessen sker i en **fast rækkefølge**.
- Er valgfri, men nyttig, når vi vil **automatisk bygge standardkonfigurationer**.
- Bruges hvis der er Afhængighed.

Kræver udvidelse, hvis parametre skal sættes

- F.eks kunne name, class og health være params.

```
public class CharacterDirector
{
    private ICharacterBuilder builder;
    public CharacterDirector(ICharacterBuilder builder)
    {
        this.builder = builder;
    }
    public Character Construct()
    {
        return builder
            .SetName("Warrior King")
            .SetClass("Warrior")
            .SetHealth(150)
            .Build();
    }
}
```

Builder

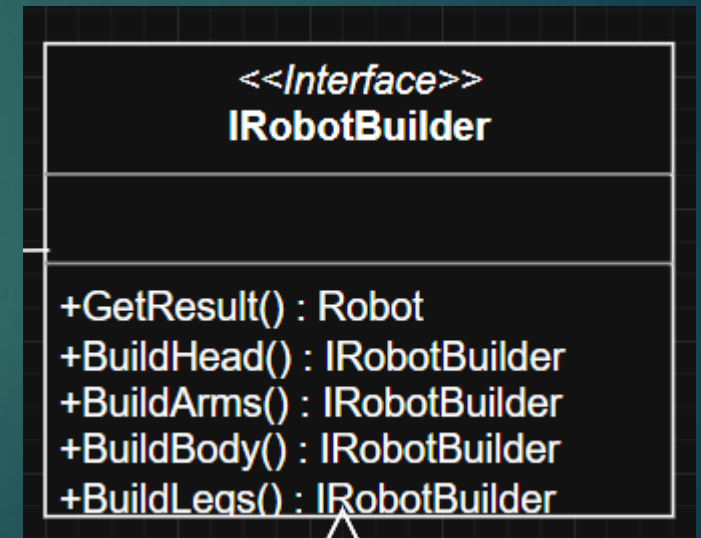
- ▶ Robot eksempel:
 - ▶ Product = Robot
 - ▶ Kan bygges step for step
 - ▶ Stadig være funktionel, uden nogle af delene

Robot
-head : string -arms : string - body : string - legs: string
+SetHead(headString : string) : void +SetArms(armsString : string) : void +SetBody(bodyString : string) : void +SetLegs(legsString : string) : void



Builder

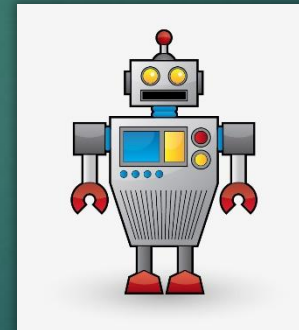
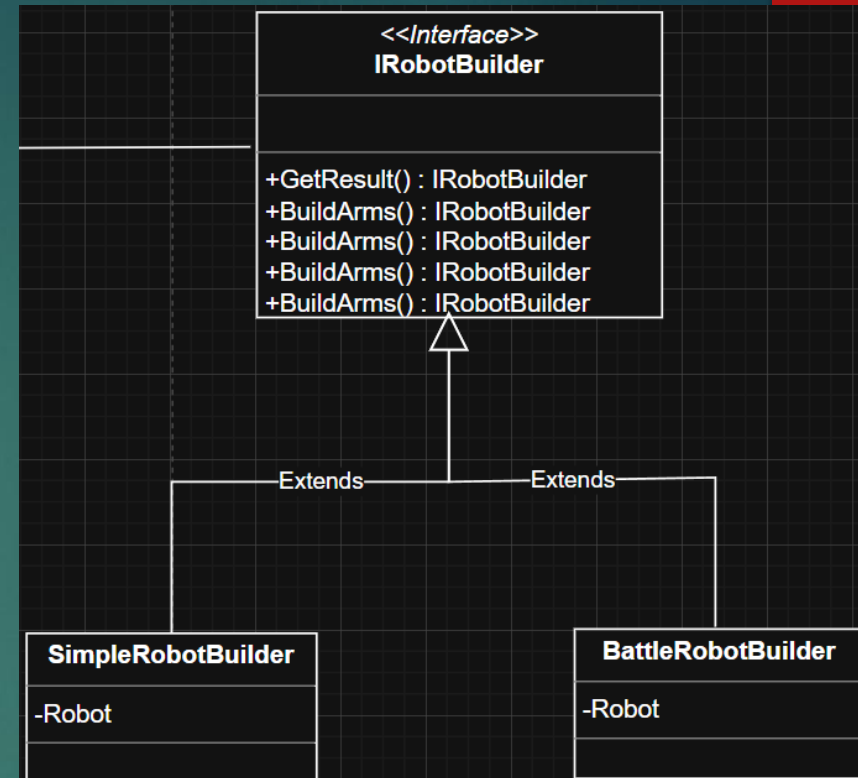
- ▶ IBuilder:
 - ▶ De forskellige dele robotten kan have
 - ▶ HUSK: returnerer selve interface typen
 - ▶ Giver support for fluent Interface



Builder

- ▶ ConcreteBuilder:
 - ▶ Implementer interface.
 - ▶ Example BuildHead():

```
public IRobotBuilder BuildHead(string headString)
{
    robot.SetHead = headString;
    return this;
}
```



Builder

- ▶ Director
 - ▶ Bygger en robot baseret på builder typen, den får som input i sin constructor.
 - ▶ Sikre at rækkefølgen bliver overholdt!

RobotDirector

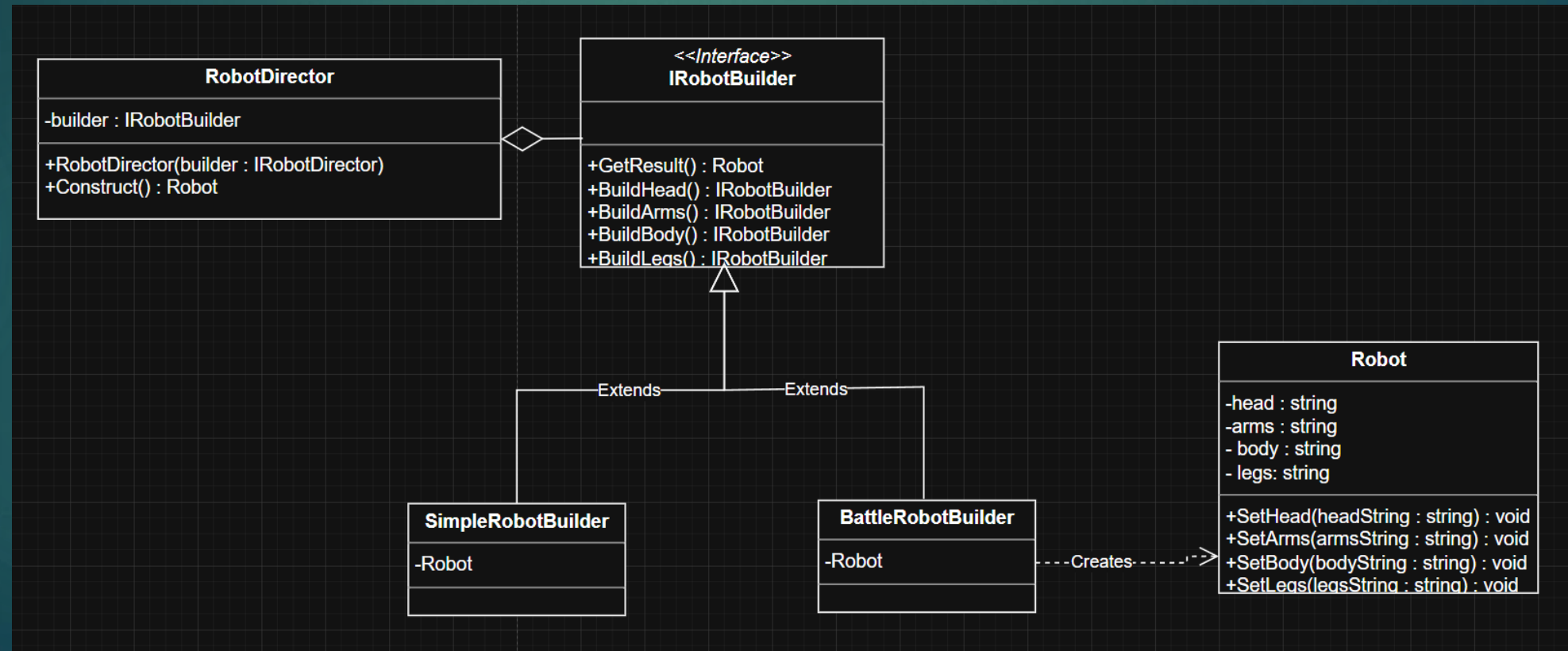
-builder : IRobotBuilder

+RobotDirector(builder : IRobotDirector)

+Construct() : Robot

Builder

► Overblik over helheden



Builder

- ▶ Exempel på output:

```
Head: High tech head
Arms: Arms with rocket launchers
Body: A body with armor plates
Legs: Unbreakable legs

-----

Head: Tin Head
Arms: Two Simple Arms
Body: A simple body with 1 button(on/off)
Legs: Two tin legs
```

Builder

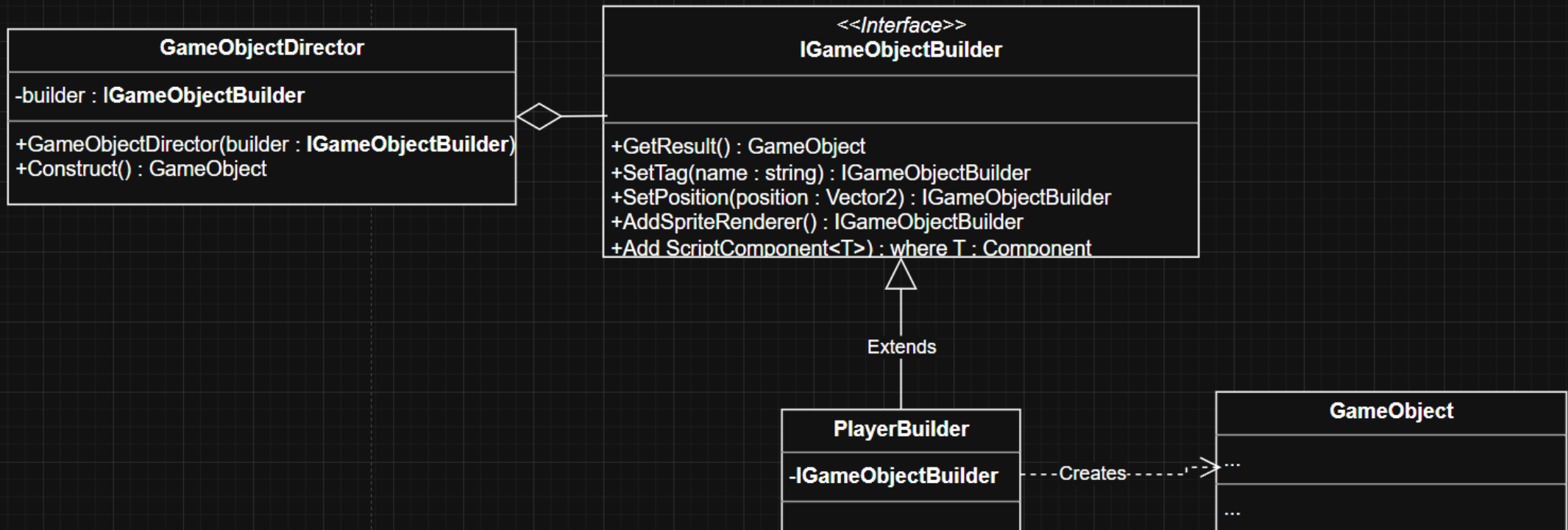
- ▶ Vi kan gøre brug af et builder pattern til at bygge vores objekter.
 - ▶ Pt. Bygger vi vores Player i GameWorld.
 - ▶ Dette kan blive meget rodet når vi senere skal tilføje flere objekter eller components.

```
GameObject playerGo = new GameObject();  
Player player = playerGo.AddComponent<Player>();  
playerGo.AddComponent<SpriteRenderer>();  
gameObjects.Add(playerGo);
```

GameWorld.cs

Builder

- ▶ Vi kan lave et builder pattern med en PlayerBuilder, som ser ud på følgende måde:

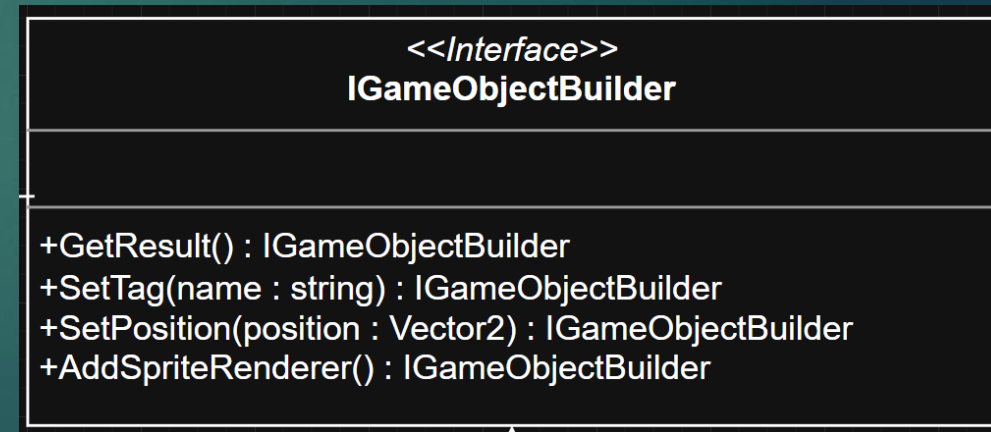


Builder – generisk add.

- ▶ Interfacet kan nemt udvides, hvis nye typer er relevante for alle GameObjecter ved opretelse.
- ▶ SpriteRenderer er her sat til at blive added som sin egen type, da der dermed vil være muligt at sætte værdier på komponenten efter behov.
- ▶ Components kan sættes generisk:

```
public IGameObjectBuilder AddScriptComponent<T>() where T : Component
{
    gameObject.AddComponent<T>();
    return this;
}
```

- ▶ Her kan vi dog ikke sætte parametre!



Builder

- ▶ Når vi har implementeret en builder pattern kan vi tilføje vores player til spilet på følgende måde.
 - ▶ Director kan instantieres med en anden type builder hvis man vil skabe andre objekter.

```
GameObjectDirector director = new GameObjectDirector(new PlayerBuilder());  
GameObject playerGo = director.Construct();  
gameObjects.Add(playerGo);
```

GameWorld.cs

ØVELSE 13

Løsning
<https://www.youtube.com/watch?v=nCMN74pmfyY>

- ▶ Gør brug af et builder pattern til at bygge jeres Player med de components der er behov for.
 - ▶ Dvs:
 - ▶ SpriteRenderer
 - ▶ Player
 - ▶ Sørg for at player er tagget med "player"
 - ▶ Sørg for at Player Builderen står for at binde Player'ens commands

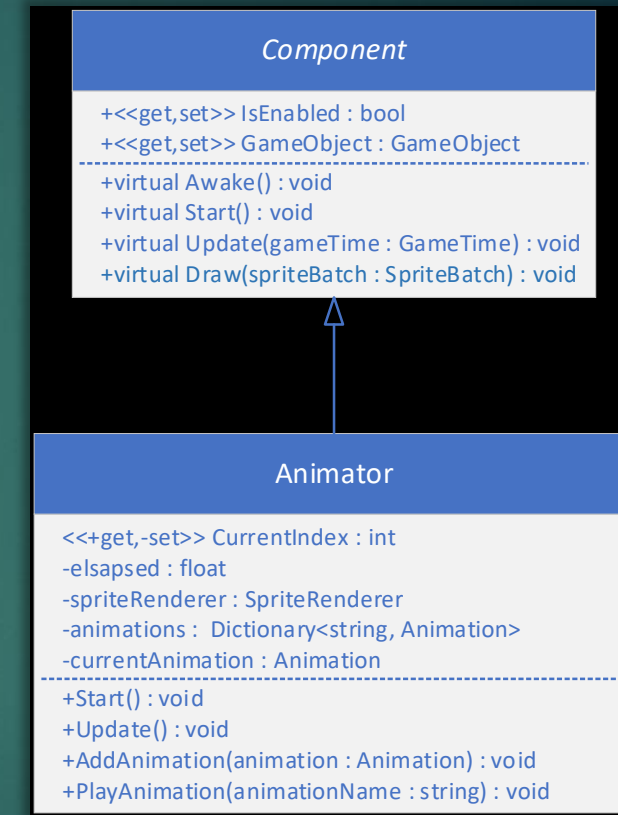
Undervisning
<https://www.youtube.com/watch?v=vnbSoGjPg4U>

ANIMATOR

BLOT EN COMPONENT 😊

ANMATOR

- ▶ Vi vil gerne kunne animere objekter i vores spil.
- ▶ Vi kan oprette et component til at håndtere dette.
- ▶ Dette component kan være en Animator.
- ▶ En Animator component, animerer et GameObject ved at ændre dens sprite, som en SpriteRenderer viser.
 - ▶ Dvs. animator har **afhængighed** af SpriteRenderer.



ANIMATOR

► Fields

- CurrentIndex: Hvilken sprite skal animatoren vise.
- Elapsed: Hvor lang tid (i sekunder) er der gået siden sidste frame.
- spriteRenderer: En reference til objektets spriteRenderer.
- Animations: En dictionary med objektets Animationer.
 - Animation er en anden klasse.
- currentAnimation: Den animation der afspilles i øjeblikket.

Animation

```
<<+get,-set>> FPS : float  
<<+get,-set>> Name : string  
<<+get, set>> Sprites : Texture2D[]  
-----  
+Animation(name : string, sprites : Texture2D[], fps : float )
```

Animator

```
<<+get,-set>> CurrentIndex : int  
-elapsed : float  
-spriteRenderer : SpriteRenderer  
-animations : Dictionary<string, Animation>  
-currentAnimation : Animation  
-----  
+Start() : void  
+Update() : void  
+AddAnimation(animation : Animation) : void  
+PlayAnimation(animationName : string) : void
```

ANIMATOR

► Metoder

- Start(): Opretter en reference til objektet's SpriteRenderer. Ved brug af GetComponent.
- Evt Tjek og fejl hvis ingen spriteRenderer er på GameObjectet.
- Update(): Animerer objektet.

```
public override void Update(GameTime gameTime)
{
    timeElapsed += GameWorld.Instance.DeltaTime;

    CurrentIndex = (int)(timeElapsed * currentAnimation.FPS);

    if (CurrentIndex > currentAnimation.Sprites.Length - 1)
    {
        timeElapsed = 0;
        CurrentIndex = 0;
    }
    spriteRenderer.Sprite = currentAnimation.Sprites[CurrentIndex];
    base.Update(gameTime);
}
```

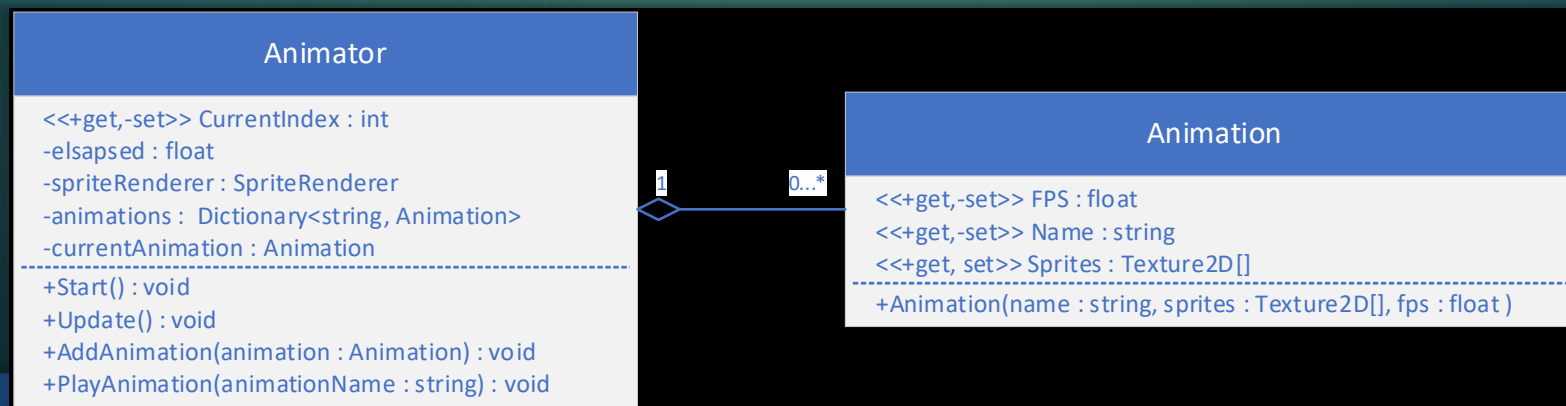
Animator

```
<<+get,-set>> CurrentIndex : int
-elapsed : float
-spriteRenderer : SpriteRenderer
-animations : Dictionary<string, Animation>
-currentAnimation : Animation
-----
+Start() : void
+Update() : void
+AddAnimation(animation : Animation) : void
+PlayAnimation(animationName : string) : void
```

- AddAnimation(Animation animation) tilføjer en Animation til animations.
- PlayAnimation(string animationName): Afspiller en ny animation ved at ændre current Animation

ANIMATOR

- ▶ En animator benytter sig af instanser af klassen Animation til at afspille animationer.
 - ▶ En simpel container klasse.
- ▶ Fps: Hvor hurtigt skal animationen afspilles.
- ▶ Name: Navnet på animationen.
- ▶ Sprites: Et array af alle sprites der tilhører animationen.



ØVELSE 14

Løsning
https://youtu.be/ALal4k6_Ydo

- ▶ Lav en PlayerDirector, der bygger en Player med SpriteRenderers, Animator og Animations.
- ▶ Flyt fælles funktionalitet fra PlayerBuilder til BaseGameObjectBuilder, så den kan genbruges af andre GameObjects.
- ▶ Udvid PlayerBuilder til at håndtere Animator og Animations.
- ▶ Sørg for, at Animator kun kan tilføjes, hvis SpriteRenderer allerede er der.
- ▶ Tilføj en Forward animation (1 fwd, 2 fwd, 3 fwd, 4 fwd), der looper.
- ▶ Når I er færdige, skal GameObjectDirector kun bruges til generelle objekter, mens PlayerDirector håndterer Players.



Undervisning
<https://www.youtube.com/watch?v=3iGazVMcA3Y>

OBSERVER

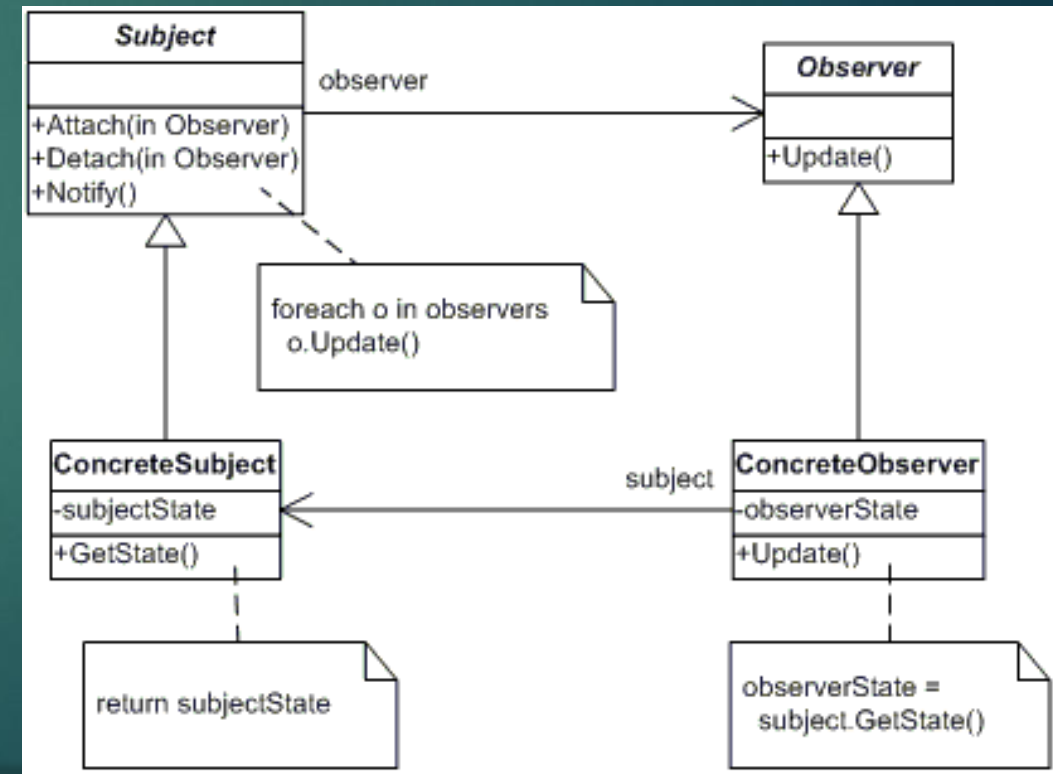


OBSERVER PATTERN

- ▶ Formål:
 - ▶ Bruges til at informere andre objekter om at en hændelse er sket.
- ▶ Bruges f.eks.
 - ▶ I et achievement system, til at lave en notifikation når spiller gør noget der kan udløse et achievement.
 - ▶ Til opdatere en GUI.
 - ▶ F.eks Når en spiller er død

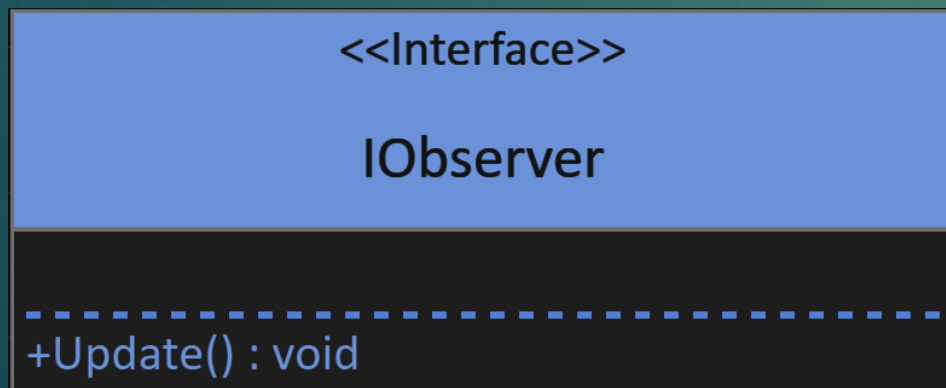
OBSERVER PATTERN

- ▶ Strukturen ser ud på følgende måde:
- ▶ Observere observerer deres subject.
- ▶ Dvs. subject er hvilken type event det er!
 - ▶ Lav derfor interface til hver type event.
 - ▶ Gør det klart hvilke klasser kan observeres
 - ▶ Og hvilke kan modtage opdateringer



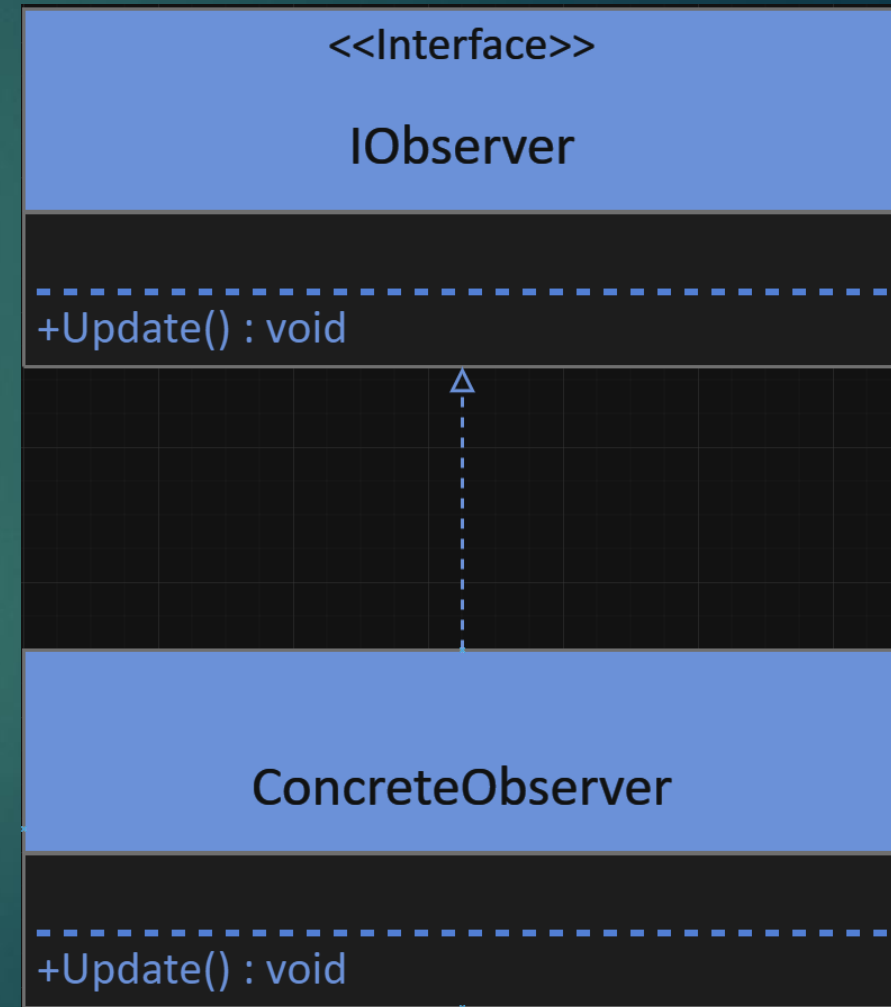
OBSERVER PATTERN

- ▶ Et Observer interface
 - ▶ Definerer en Update metode, som bruges til at notificere alle der lytter til et bestemt event.
 - ▶ Man kan selvfølgelig udvide her, hvis notify skal have parametre
 - ▶ (husk også at lave et tilsvarende Subject)



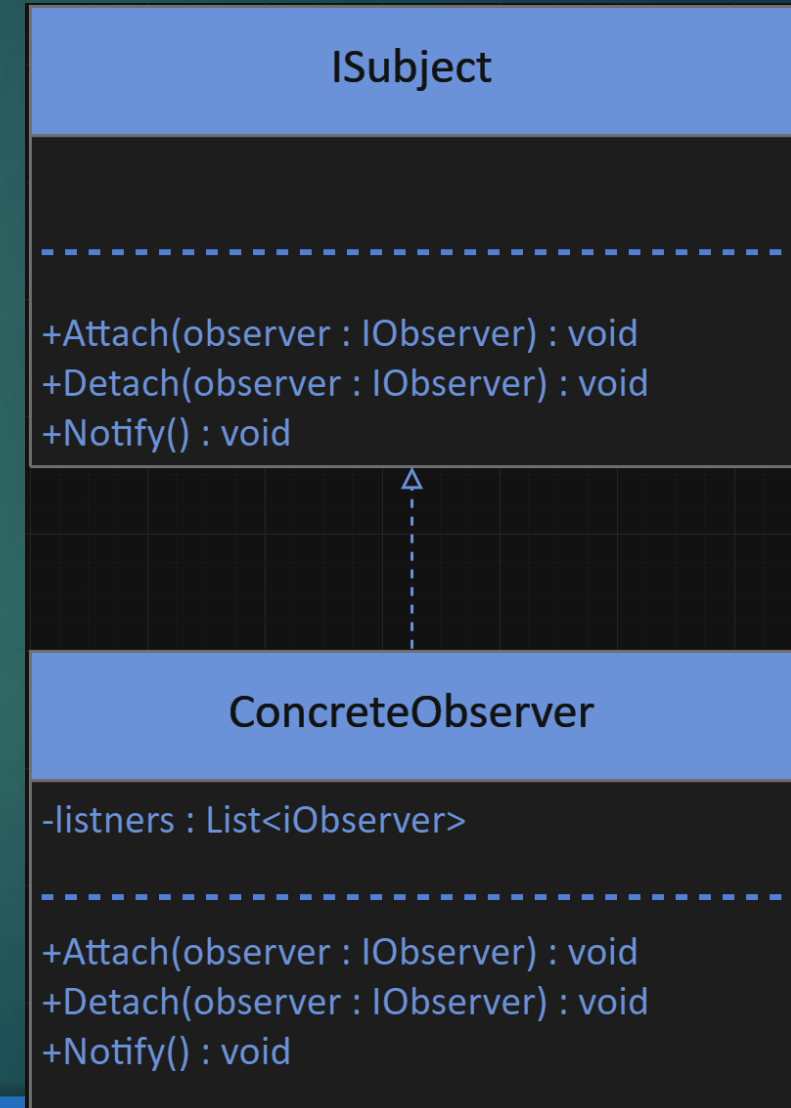
OBSERVER PATTERN

- ▶ En implementering af en observer.
 - ▶ Laver sin egen implementering af Update.
 - ▶ Update bestemmer hvad der skal ske når et subject trigger Notify.
 - ▶ F.eks.
 - ▶ Unlock et achievement.
 - ▶ Opdatere UI
- ▶ Da det blot er et interface kan vi tilføje IObserver til eksisterende klasser.



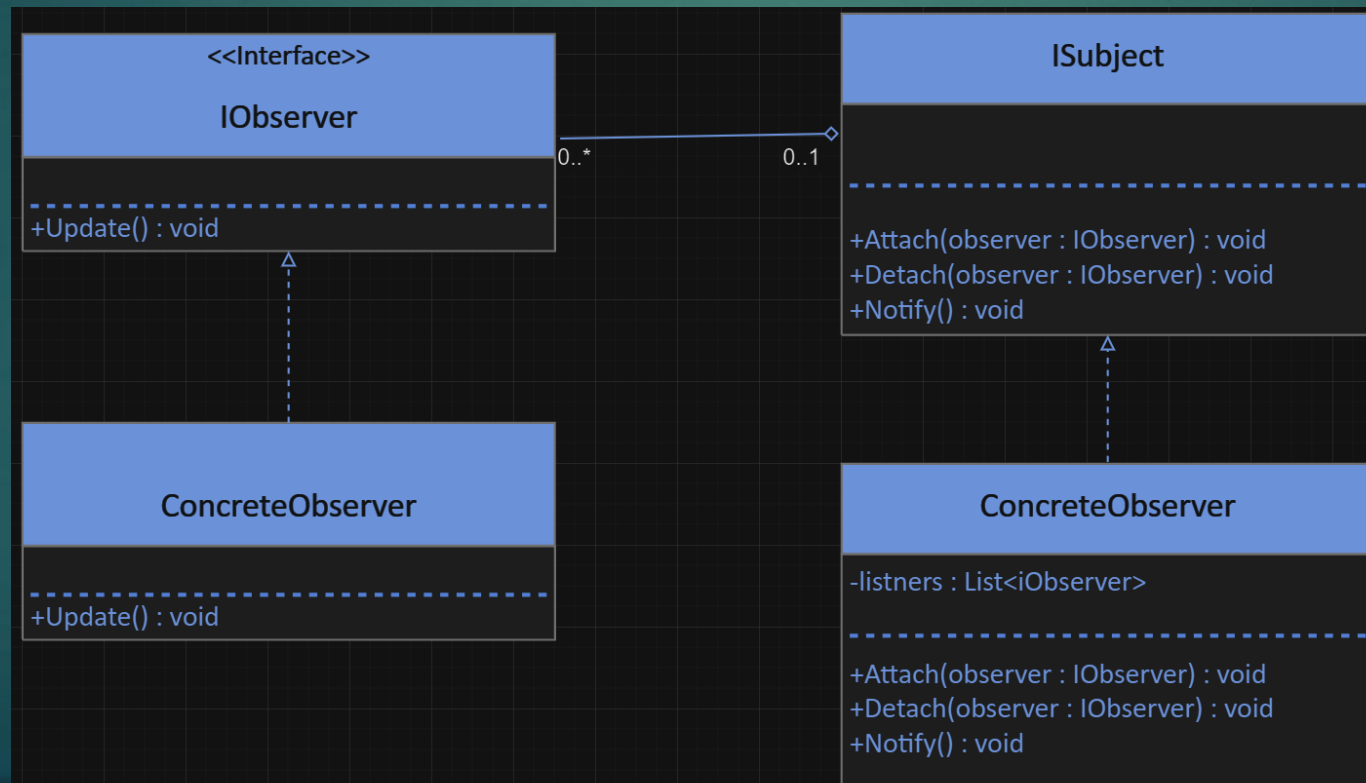
OBSERVER PATTERN

- ▶ Vi laver et interface der giver signatur til at tilføje og fjerne en observer
- ▶ I den konkrete observer gemmer vi så en liste af observers, og kalder Update på alle iObservers i listen
- ▶ Da det er et interface, kan vi tilføje dette til alle der kan invoke et event af denne type
 - ▶ Det kunne f.eks være gameWorld



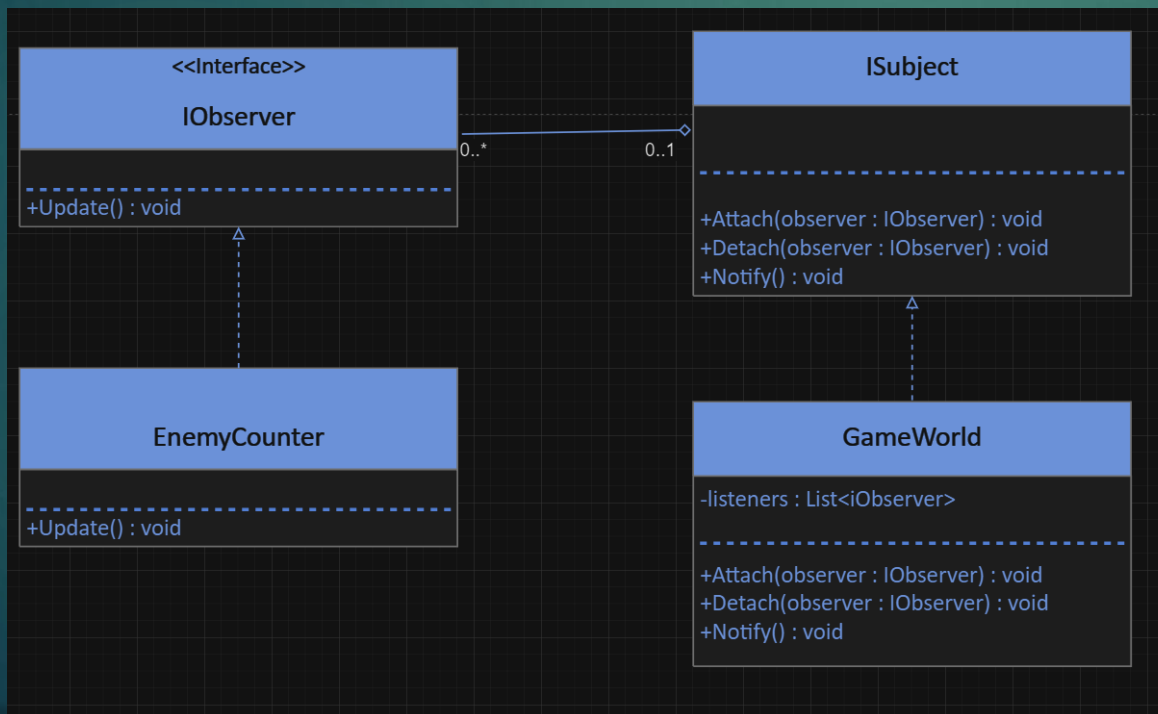
OBSERVER PATTERN

- ▶ En implementering kan se ud på følgende måde.
 - ▶ Example er den klasse der skal reagere på et event.



OBSERVER PATTERN

- ▶ Kan f.eks sættes op således:
- ▶ Hvor gameworld selvfølgelig har implementeret ISubject interfacet.



```
public class EnemyCounter : Component, IObserver
{
    private string text;
    private SpriteFont font;
    0 references
    public EnemyCounter(GameObject gameObject) : base(gameObject)
    {
    }
    0 references
    public EnemyCounter(GameObject gameObject, string initialText) : base(gameObject)
    {
        this.text = initialText;
        GameWorld.Instance.Attach(this);
    }

    3 references
    public override void Awake()
    {
        base.Awake();
        font = GameWorld.Instance.Content.Load<SpriteFont>("text");
    }

    2 references
    public override void Draw(SpriteBatch spriteBatch)
    {
        if (!string.IsNullOrEmpty(text))
        {
            spriteBatch.DrawString(font, text, Vector2.Zero, Color.White);
        }
    }

    int enemiesSpawned= 0;
    2 references
    public void Update()
    {
        enemiesSpawned++;
        text = enemiesSpawned.ToString();
    }
}
```


ØVELSE 15

Løsning

https://www.youtube.com/watch?v=8KMNoqGdc_0

- ▶ Lav en Enemy counter component
- ▶ Opret et nyt gameObject der har en enemy counter component
 - ▶ Denne skriver blot ud hvor mange enemies der er blevet spawnet
 - ▶ Denne bruger fonts i monogame:
<http://rbwhitaker.wikidot.com/monogame-drawing-text-with-spritefonts>
- ▶ Lad Gameworld Være Subject og Enemy counter være Observer
- ▶ Sørg for at når en enemy spawner den Notify'er Gameworld'en.

